SMART WASTE MANAGEMENT WEB APPLICATION

Submitted To
SIR WAHAJ ALI

Submitted By
ARSHIA BADAR

F23BINFT1M01058

Department of Information Technology
Faculty of Computing

The Islamia University of Bahawalpur

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Product Perspective 4c
1.4 User Characteristics 4
1.5 Similar Apps and Systems / Literature Review 4
1.6 Proposed Technologies 5
2. Requirements 5
2.1 Functional Requirements 5
2.2 Non-Functional Requirements 7
3. Data Representation 7
3.1 Use Cases and Flow of Processes 7
3.2 Sequence Diagram 9
3.3 Activity Diagram 10

1. Introduction

The Smart Waste Management Web Application is a modern, intelligent web-based system designed to help cities and municipal waste management departments monitor waste bins and manage waste collection operations efficiently. The system addresses the critical urban challenge of waste overflow and inefficient collection scheduling by integrating smart tracking, automated scheduling, and route optimization into a single unified platform.

The application enables real-time monitoring of bin fill levels, automated generation of collection schedules based on bin status, and optimized route assignment for waste collection drivers. Citizens can submit complaints directly through the platform, while administrators gain a centralized dashboard for comprehensive oversight and reporting. By combining web technology with data-driven decision making, the system reduces fuel costs, prevents overflow situations, and improves overall city cleanliness.

1.1 Purpose

The purpose of the Smart Waste Management Web Application is to develop a smart and user-friendly system that helps municipal departments and city administrators monitor waste bins in real time, automate collection scheduling, and optimize driver routes. The system is created to improve urban cleanliness and operational efficiency by replacing manual, reactive waste collection methods with a proactive, data-driven approach.

This documentation explains how the system works, what technologies are used, which features are included, and how the waste management process is performed. It helps developers, supervisors, testers, drivers, and administrators clearly understand the objectives and functionality of the project. The project focuses on reducing overflow incidents, minimizing fuel consumption, and enabling transparent complaint management between citizens and waste authorities.

1.2 Scope

The scope of the Smart Waste Management Web Application covers the development of a web-based system that manages waste bin monitoring, collection scheduling, route optimization, complaint handling, and administrative reporting. The system serves three primary user groups: administrators who manage the entire system, drivers who execute collection routes, and citizens who monitor bin status and submit complaints.

The system will provide the following core functionalities:

- Real-time monitoring of waste bin fill levels across the city
- Automated and manual waste collection schedule generation
- Driver route assignment and optimization based on bin priority
- IoT sensor simulation for bin fill-level updates
- Citizen complaint submission and admin resolution tracking
- Admin dashboard with statistics and performance overview
- Secure role-based login for Admin, Driver, and Citizen

1.3 Product Perspective

The Smart Waste Management Web Application is designed as a modern web application that combines real-time data management with intelligent scheduling to provide efficient waste collection operations. The system works through a role-based web interface where administrators monitor bins and manage schedules, drivers view their assigned routes, and citizens submit and track complaints.

The application is developed using a modular structure that includes the frontend, backend, and data persistence components. The frontend is responsible for role-specific dashboards and user interaction, while the Flask backend manages requests, validates data, and controls the scheduling and routing logic. JSON files provide lightweight data storage suitable for the prototype stage.

The system is flexible and can be improved in the future by integrating real IoT sensor hardware, a relational database, and cloud deployment. New features such as real-time SMS notifications, GPS-based tracking, and predictive analytics can be added without restructuring the core system.

1.4 User Characteristics

The system is designed for three distinct types of users with different roles and permissions:

User Role	Access Level	Key Responsibilities
Administrator	Full Access	Manage bins, schedules, routes, drivers, complaints, and reports
Driver / Waste Collector	Limited Access	View assigned collection routes and update task status
Citizen / User	Public Access	Register, log in, submit complaints, and check bin status

- Primary users: municipal waste departments, waste collectors, city administrators, and citizens
- Technical level: the system is designed for users with basic computer and internet skills
- Accessibility: clear fonts, high contrast, mobile-responsive layout, and simple navigation

1.5 Similar Apps and Systems / Literature Review

Several web-based and IoT-enabled smart waste management systems have been developed to address urban waste challenges. An examination of existing solutions reveals their strengths and gaps that this system aims to address.

- Bigbelly Smart Waste System uses solar-powered sensors to monitor bin fill levels and send real-time alerts, demonstrating the effectiveness of sensor integration for reducing unnecessary collection trips.
- Enevo ONE is a waste analytics platform that uses ultrasonic sensors and predictive algorithms to optimize collection routes, highlighting the value of data-driven scheduling.

- SmartBin (Ireland) provides fill-level monitoring and automated notifications to waste operators, showing how web dashboards can improve municipal response time.
- RecycleSmart (Australia) offers citizen-facing apps for reporting overflow and tracking collection schedules, which demonstrates the importance of public engagement in waste management.

Lessons from these systems highlight the importance of real-time data visibility, role-based access, automated scheduling, and citizen participation. The Smart Waste Management Web Application incorporates these principles within a lightweight, prototype-level web system using Flask and JSON-based storage, making it accessible for academic development and future scaling.

1.6 Proposed Technologies

The system is built using the following technology stack:

Backend Framework: Python with Flask
Frontend: HTML5, CSS3, JavaScript (inline templates using Flask render_template_string)
Data Storage: JSON files (drivers.json, bins.json, routes.json, complaints.json, users.json)
Authentication: Werkzeug password hashing using generate_password_hash() and check_password_hash()
Session Management: Flask session with secret key
Deployment: Localhost using XAMPP / Python development server for prototype stage
Development Tool: Visual Studio Code

2. Requirements

The following sections describe the functional and non-functional requirements of the Smart Waste Management Web Application.

2.1 Functional Requirements

User Registration and Login

The system provides role-based authentication for three user types:

- Admin: logs in with fixed credentials (username: admin, password: admin123) and gains access to the full Admin Panel
- Driver: logs in using pre-configured driver credentials stored in drivers.json and is directed to the Driver Dashboard
- Citizen: registers a new account with full name, email address, and password; logs in to access complaint submission and bin status features
- Validation: empty fields, invalid email formats, short passwords, and duplicate email registrations are all rejected with appropriate error messages
- Session management: each role is assigned a separate session key (admin, driver, citizen) to maintain secure access control

Smart Bin Monitoring

- The system displays all registered waste bins with their unique ID, location, fill percentage, and current status
- Bin status is automatically calculated: bins with fill level >= 80% are marked FULL, >= 50% are MEDIUM, and below 50% are NORMAL

- Status is visually represented using color-coded badges: red for FULL, amber for MEDIUM, and green for NORMAL
- A progress bar visualization shows the fill level for each bin on the monitoring page
- Admin can view all bins from the Bin Monitoring page in a responsive card grid layout

IoT Sensor Input

- The system provides an IoT Sensor Input page where simulated sensor readings can be entered manually
- Admin enters a bin ID and a new fill level value; the system updates the bin record and recalculates status automatically
- This feature simulates real-world IoT sensor integration in the absence of physical hardware

Waste Collection Scheduling

- Auto Schedule Generation: the system scans all bins with FULL status and automatically creates collection tasks, assigning each full bin to an available driver in round-robin order
- Manual Schedule Assignment: admin can manually assign a driver, route description, pickup bin, bin location, and destination for custom scheduling
- Schedules are persisted in routes.json and remain available for driver reference and admin review

Route Optimization

- The Routes page displays all assigned collection tasks sorted by bin fill level in descending order, prioritizing the most critical bins first
- Each route record shows driver name, route description, pickup bin ID, bin location, destination, fill percentage, and status
- This fill-level-based sorting serves as the route optimization algorithm, ensuring drivers address the highest-priority bins first

Admin Dashboard

- The dashboard displays summary statistics including total bins, number of full bins, medium bins, normal bins, total drivers, total routes, and total complaints
- A current bin status table is shown on the dashboard for quick overview
- Admin can navigate to all system sections from the sidebar: Bin Monitoring, IoT Sensor Input, Auto Schedule, Manual Assign, Routes, Add Driver, and Complaints

Complaint Management

- Citizens can submit complaints by providing their name, complaint category (overflowing bin, missed collection, damaged bin, other), location, and description
- The system records all complaints with a timestamp and assigns them a Pending status
- Admin can review all complaints in a table view showing citizen name, category, location, description, time, and current status
- Admin can update complaint status to In Progress or Resolved directly from the complaints management page

Driver Management

- Admin can add new drivers by entering a driver name and password through the Add Driver form
- Driver credentials are stored with hashed passwords in drivers.json

- Driver Dashboard shows the driver all routes assigned to them with full pickup and destination details

2.2 Non-Functional Requirements

Reliability and Performance

The system must respond quickly to all user actions, accurately update bin status based on fill-level input, and handle concurrent users accessing different role dashboards without performance degradation. JSON file operations are performed synchronously and are optimized for the prototype scale.

Usability and Accessibility

The interface must be clean, intuitive, and easy to navigate for users with minimal technical knowledge. The application uses a consistent dark-themed design with clear sidebar navigation, color-coded status indicators, and responsive layout so it runs smoothly on desktop and mobile browsers.

Security and Privacy

All passwords are stored using Werkzeug's secure hashing functions and are never stored in plain text. Session-based authentication ensures that users can only access features permitted for their role. Direct URL access to restricted pages redirects unauthorized users to the appropriate login page.

Scalability and Extensibility

The system is structured to allow future migration from JSON file storage to a relational database such as MySQL. The modular route and scheduling logic can be extended with real IoT sensor APIs, GPS tracking, and advanced route optimization algorithms without requiring a redesign of the core system architecture.

System Availability

The prototype is designed for localhost deployment using the Python Flask development server or XAMPP. For production use, the system can be deployed on a cloud server with appropriate uptime guarantees.

3. Data Representation

3.1 Use Cases and Flow of Processes

Use case Diagram(figure 3.1) formally represent the process flows defined by the functional requirements of the Smart Waste Management Web Application.

System: Smart Waste Management System

Actors:

- Citizen
- Admin
- Driver / IoT Sensor

Use Cases:

1.Register/ Login: Users create an account or log in to access the system features. Citizens and admins use authentication to access dashboards.

2.Monitor Bins: Citizens and admins can check bin status, fill levels, and monitor waste conditions.

3.Submit Complaint: Citizens can report issues related to waste collection, overflowing bins, or cleanliness problems.

4.Assign Schedule: Admin assigns collection schedules and tasks to drivers for waste collection.

5.Auto-Generate Schedule: The system automatically creates optimized schedules for waste collection based on requirements.

6.Collect Waste: Drivers perform waste collection tasks from assigned locations and update task status after completion.

7.Send Sensor Data: IoT sensors send bin fill-level information and waste status data to the system for monitoring.

8.Manage Complaints: Admin reviews, manages, and resolves complaints submitted by citizens.

Actor Responsibilities:

Citizen:

- Register/Login
- Monitor bins
- Submit complaints

Admin:

- Register/Login
- Monitor bins
- Assign schedules
- Auto-generate schedules
- Manage complaints

Driver / IoT Sensor:

- Collect waste
- Send sensor data

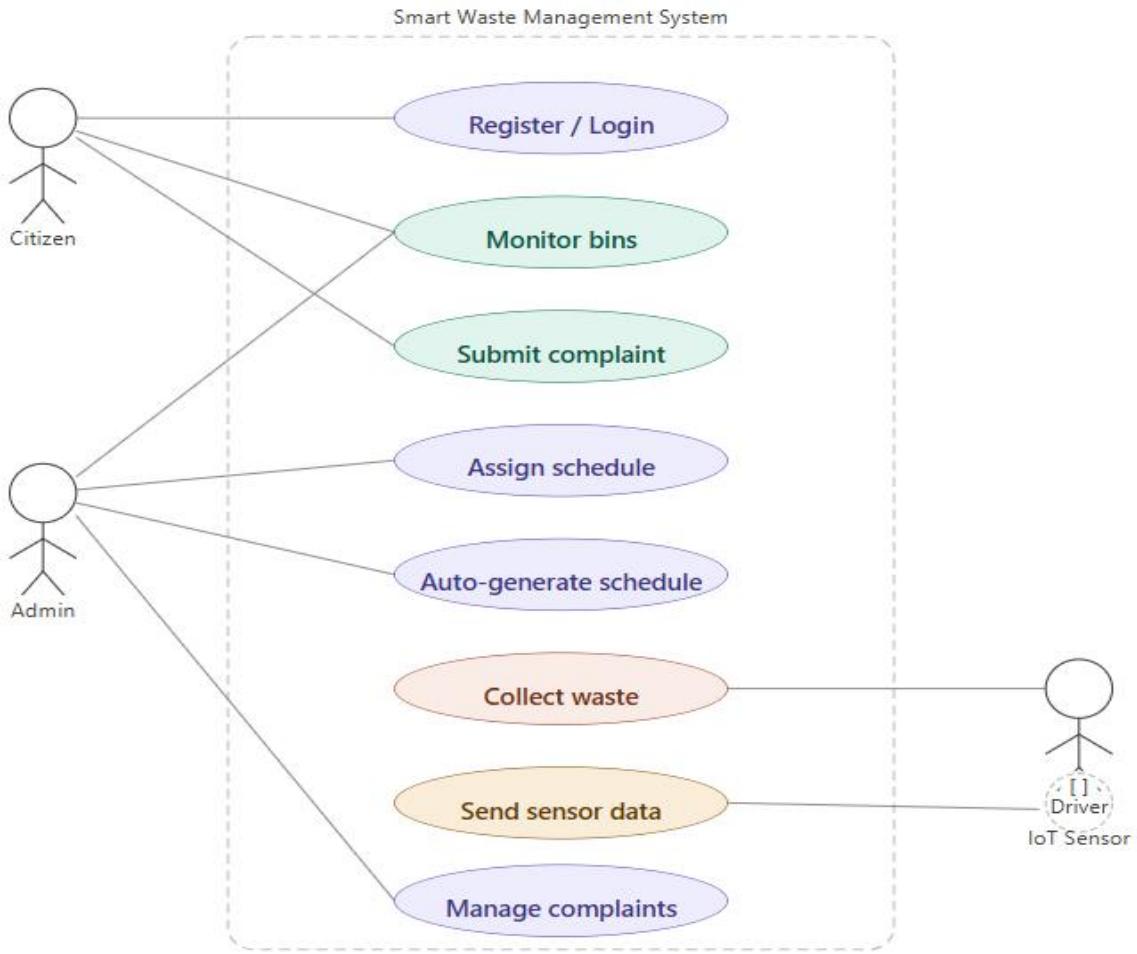


Figure 3.1 Use case Diagram

3.2 Sequence Diagram

The sequence diagram, shown in Figure 3.2, illustrates the interaction between users, administrators, the web interface, and the backend system during key operations of the Smart Waste Management Web Application. For user registration, a citizen accesses the registration page, submits personal details, and the Flask backend validates the information by checking existing records in the user database. If the email is unique and validation succeeds, the password is securely hashed, the new user record is stored, and the citizen is redirected to the login page; otherwise, an error message is displayed. During login, users such as citizens, drivers, or administrators access their respective login pages and submit credentials. The backend verifies the credentials against stored records, creates a role-specific session upon successful authentication, and redirects the user to the appropriate dashboard, while invalid credentials result in an error message and retention on the login page. For automatic schedule generation, the administrator accesses the scheduling module, where the system retrieves waste bin records and identifies bins marked as full. Upon the administrator's request to generate schedules, the system loads available driver information, assigns collection tasks to drivers using a round-robin allocation mechanism, stores the generated routes, and displays a confirmation message along with the newly created task assignments. This sequence demonstrates the flow of requests and responses between actors, the web interface, the Flask application, and the underlying data storage components.

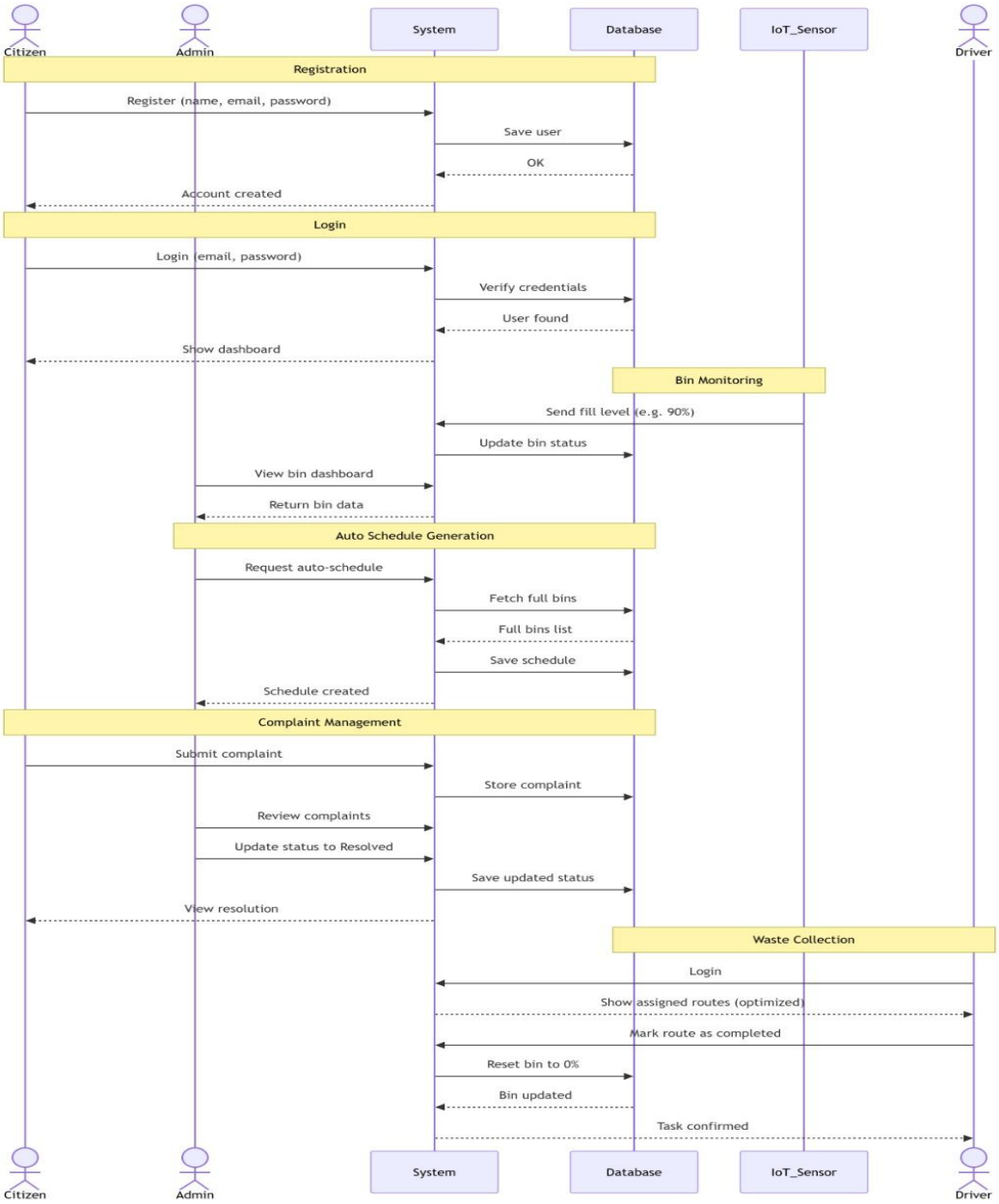


Figure 3.2 Sequence Diagram

3.3 Activity Diagram

The activity diagram, shown in Figure 3.3, illustrates the overall workflow of the Smart Waste Management Web Application across the three primary user roles: Admin, Driver, and Citizen. The process begins when an administrator logs into the system and is authenticated by the backend before being redirected to the Admin Dashboard. From there, the administrator monitors waste bin statuses, updates fill-level information received from IoT sensors when necessary, and the system automatically recalculates each bin's status based on the updated data. The administrator can then initiate the auto-schedule generation process, where the system identifies full bins, assigns collection tasks to available drivers, and creates route records. The administrator subsequently reviews generated routes, manages driver assignments, addresses citizen complaints, and logs out of the system. In the driver workflow, the driver logs in using assigned credentials, accesses the Driver Dashboard after successful authentication, views assigned collection routes, performs waste collection activities according to route instructions, and then logs out. For citizens, the workflow begins with account

registration or login, followed by access to the Citizen Portal where bin statuses can be viewed. Citizens may submit complaints by providing relevant details such as category, location, and description, after which the system stores the complaint with a pending status for administrative review. Once their activities are completed, citizens log out of the application. The activity diagram therefore represents the sequence of actions, decision points, and interactions that collectively support waste monitoring, collection scheduling, and citizen engagement within the system.

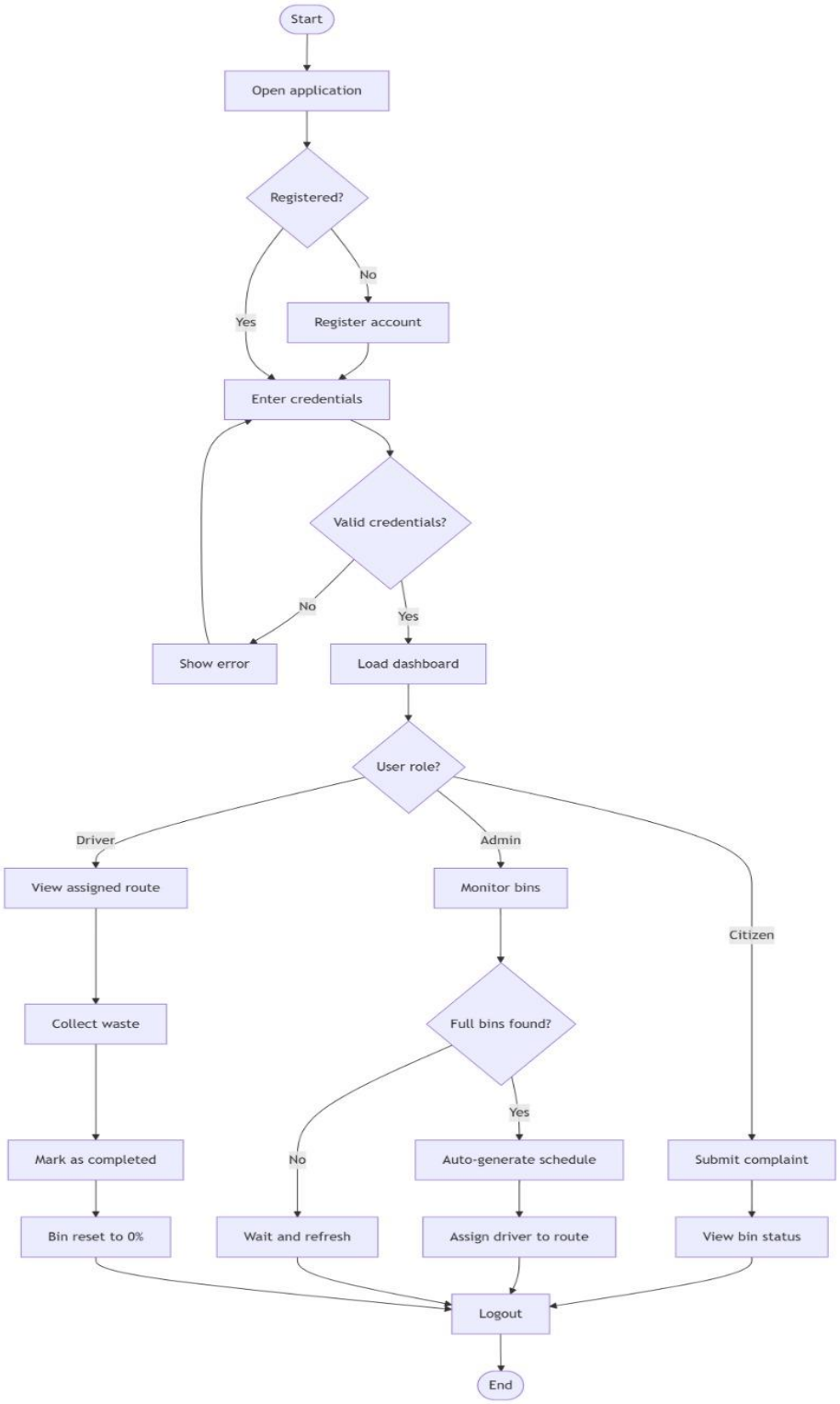


Figure 3.3 Activity Diagram